

Penalized Regression for Psychological Data

Nathaniel E. Helwig

Assistant Professor of Psychology and Statistics
University of Minnesota (Twin Cities)



I/O Psychology Talk
March 21, 2016

Outline of Talk

1) Regression Reminder

- Motivation
- Overview

2) Ridge Regression

- Motivation
- Overview

3) Lasso Regression

- Motivation
- Overview

4) Elastic Net Regression

- Motivation
- Overview

5) Smoothing Spline ANOVA

- Motivation
- Overview

6) Student Performance Example

- Data Overview
- Method Comparison

Regression Reminder

Regression Background

A **regression model** relates the response variable Y to the independent variables $X_1, \dots, X_p$ using some systematic rule.

For example, the **general linear model (GLM)** assumes that

$$y_i = \beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip} + \epsilon_i$$

for $i = 1, \dots, n$ where

- y_i is the observed response variable for the i -th subject
- x_{ij} is the j -th observed predictor for the i -th subject
- $(\beta_0, \beta_1, \dots, \beta_p)$ are the unknown regression coefficients
- $\epsilon_i \stackrel{\text{iid}}{\sim} (0, \sigma^2)$ is independent, identically distributed measurement error

Implies that $(y_i | \mathbf{x}_i) \stackrel{\text{ind}}{\sim} (\beta_0 + \sum_{j=1}^p \beta_j x_{ij}, \sigma^2)$

Regression and Prediction

In most regression problems, the focus is **prediction**

- Modeling the expected value of Y given some $X_1, \dots, X_p$
- Want to predict $E(Y|X_1, \dots, X_p)$ for different predictor values

The general regression procedure is

- Estimate some parameters that determine systematic regression rule
- Use parameters to predict Y for new $X_1, \dots, X_p$

For example, in the GLM we are typically interested in

- Estimating the regression coefficients $(\beta_0, \beta_1, \dots, \beta_p)$
- Prediction is $\hat{Y} = \hat{\beta}_0 + \sum_{j=1}^p \hat{\beta}_j X_j$

Multiple Linear Regression Model in Matrix Form

The multiple linear regression model has the form

$$\mathbf{y} = \mathbf{1}_n\beta_0 + \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$$

where

- $\mathbf{y} = (y_1, \dots, y_n)' \in \mathbb{R}^n$ is the $n \times 1$ response vector
- $\mathbf{1}_n = \{1\}_{n \times 1}$ is an $n \times 1$ vector of ones
- $\mathbf{X} = \{x_{ij}\}_{n \times p} \in \mathbb{R}^{n \times p}$ is the $n \times p$ design matrix
- $\boldsymbol{\beta} = (\beta_1, \dots, \beta_p)' \in \mathbb{R}^p$ is the $p \times 1$ vector of slope coefficients
- $\boldsymbol{\epsilon} = (\epsilon_1, \dots, \epsilon_n)' \sim (\mathbf{0}_n, \sigma^2 \mathbf{I}_n)$ is the $n \times 1$ error vector

Implies that $(\mathbf{y}|\mathbf{X}) \sim (\mathbf{1}_n\beta_0 + \mathbf{X}\boldsymbol{\beta}, \sigma^2 \mathbf{I}_n)$

Ordinary Least Squares Problem

The **ordinary least squares** (OLS) function is

$$\begin{aligned}\text{OLS}(\beta_0, \boldsymbol{\beta}) &= \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 \\ &= \|\mathbf{y} - \mathbf{1}_n \beta_0 - \mathbf{X} \boldsymbol{\beta}\|^2\end{aligned}\tag{1}$$

where $\|\mathbf{w}\| = \{\sum_{i=1}^n w_i^2\}^{1/2}$ denotes the Euclidean norm.

The coefficients that minimize the OLS function have the form

$$\begin{aligned}\hat{\beta}_0 &= \bar{y} - \sum_{j=1}^p \hat{\beta}_j \bar{x}_j \\ \hat{\boldsymbol{\beta}} &= (\mathbf{X}'_c \mathbf{X}_c)^{-1} \mathbf{X}'_c \mathbf{y}_c\end{aligned}$$

where $\mathbf{X}_c = (\mathbf{I}_n - n^{-1} \mathbf{1}_n \mathbf{1}'_n) \mathbf{X}$ and $\mathbf{y}_c = (\mathbf{I}_n - n^{-1} \mathbf{1}_n \mathbf{1}'_n) \mathbf{y}$

Standardized Regression

Motivation: β_j represents expected change in Y for one-unit change in X_j holding other predictors constant

- If X_j and X_k have different units (e.g., dollars and square feet), the scales of β_j and β_k are not comparable

Standardized regression model has the form

$$y_i^* = \sum_{j=1}^p \beta_j^* x_{ij}^* + \epsilon_i^*$$

where y_i^* and x_{ij}^* are standardized (mean zero and variance one)

- β_j^* represents expected change in Y standard deviations (SDs) for one SD change in X_j holding other predictors constant

Standardized Regression (continued)

Let $\mathbf{C} = \mathbf{I}_n - n^{-1}\mathbf{1}_n\mathbf{1}_n'$ denote a centering matrix, $\mathbf{S} = \text{diag}(s_j)$ where $s_j^2 = (n-1)^{-1} \sum_{i=1}^n (x_{ij} - \bar{x}_j)^2$, and $s_y^2 = (n-1)^{-1} \sum_{i=1}^n (y_i - \bar{y})^2$

- $\mathbf{X}_{cs} = \mathbf{CXS}^{-1}$ is standardized version of $X_1, \dots, X_p$
- $\mathbf{y}_{cs} = \mathbf{Cy}s_y^{-1}$ is standardized version of Y

The OLS solution for β^* has the form

$$\begin{aligned}\hat{\beta}^* &= (\mathbf{X}_{cs}'\mathbf{X}_{cs})^{-1}\mathbf{X}_{cs}'\mathbf{y}_{cs} \\ &= (\mathbf{S}^{-1}\mathbf{X}_c'\mathbf{X}_c\mathbf{S}^{-1})^{-1}\mathbf{S}^{-1}\mathbf{X}_c'\mathbf{y}_cs_y^{-1} \\ &= s_y^{-1}\mathbf{S}(\mathbf{X}_c'\mathbf{X}_c)^{-1}\mathbf{X}_c'\mathbf{y}_c \\ &= s_y^{-1}\mathbf{S}\hat{\beta}\end{aligned}$$

where $\hat{\beta}$ is the least squares estimate without standardizing.

Ridge Regression

Multicollinearity and OLS

When the predictors $X_1, \dots, X_p$ are uncorrelated, OLS will be stable.

- $\mathbf{X}'_c \mathbf{X}_c$ is diagonal if predictors are uncorrelated.

When the predictors in $\mathbf{X}_c$ are highly correlated, OLS has BIG problem.

- $\mathbf{X}'_c \mathbf{X}_c$ is nearly singular when predictors are highly correlated.
- Leads to unstable estimates of coefficients (bouncing betas)

Ridge regression shrinks and stabilizes the coefficient estimates.

Penalized Least Squares Problem

Ridge regression minimizes the **penalized least squares** (PLS) function

$$\begin{aligned}
 \text{PLS}_2(\beta_0, \boldsymbol{\beta}) &= \text{OLS}(\beta_0, \boldsymbol{\beta}) + \lambda \text{P}_2(\boldsymbol{\beta}) \\
 &= \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p \beta_j^2 \\
 &= \|\mathbf{y} - \mathbf{1}_n \beta_0 - \mathbf{X} \boldsymbol{\beta}\|^2 + \lambda \boldsymbol{\beta}' \boldsymbol{\beta}
 \end{aligned} \tag{2}$$

where $\text{P}_2(\boldsymbol{\beta}) = \sum_{j=1}^p \beta_j^2$ and $\lambda > 0$ is a **regularization parameter**.

The coefficients that minimize the ridge PLS function have the form

$$\begin{aligned}
 \hat{\beta}_{0(\lambda)} &= \bar{y} - \sum_{j=1}^p \hat{\beta}_{j(\lambda)} \bar{x}_{\cdot j} \\
 \hat{\boldsymbol{\beta}}_{\lambda} &= (\mathbf{X}'_c \mathbf{X}_c + \lambda \mathbf{I}_p)^{-1} \mathbf{X}'_c \mathbf{y}_c
 \end{aligned}$$

where $\mathbf{X}_c = (\mathbf{I}_n - n^{-1} \mathbf{1}_n \mathbf{1}'_n) \mathbf{X}$ and $\mathbf{y}_c = (\mathbf{I}_n - n^{-1} \mathbf{1}_n \mathbf{1}'_n) \mathbf{y}$

Understanding the Regularization Parameter

The regularization parameter $\lambda > 0$ controls the trade-off between fitting the data and shrinking the coefficients

- $\text{OLS}(\beta_0, \beta)$ measures fit of data
- $P_2(\beta)$ measures size of coefficients

As $\lambda \rightarrow 0$, we have $\hat{\beta}_\lambda \rightarrow \hat{\beta}$, where $\hat{\beta}$ is OLS estimate.

As $\lambda \rightarrow \infty$, we have $\hat{\beta}_\lambda \rightarrow \mathbf{0}$, which is a vector of zeros.

Note: unless units of $X_1, \dots, X_p$ are comparable, penalty is typically applied to the standardized regression coefficients $\beta_j^* = (s_j/s_y)\beta_j$.

An Optimal Regularization Parameter

Remember:

- As $\lambda \rightarrow 0$, we have $\hat{\beta}_\lambda \rightarrow \hat{\beta}$, where $\hat{\beta}$ is OLS estimate.
- As $\lambda \rightarrow \infty$, we have $\hat{\beta}_\lambda \rightarrow \mathbf{0}$, which is a vector of zeros.

Goal is to find a λ that provides an optimal balance between fitting the data and shrinking the coefficients.

- Mean-squared error (MSE) = variance + bias²
- As $\lambda \rightarrow 0$, the bias decreases and the variance increases
- As $\lambda \rightarrow \infty$, the bias increases and the variance decreases

Hoerl and Kennard (1970) showed that there exists some $\lambda > 0$ such that $E[\text{MSE}(\hat{\beta}_\lambda)] < E[\text{MSE}(\hat{\beta})]$.

Ordinary Cross-Validation

To choose the regularization parameter, we could consider minimizing the ordinary cross-validation (OCV) criterion

$$\text{OCV}(\lambda) = \frac{1}{n} \sum_{i=1}^n \left(y_i - \beta_{0(\lambda)}^{[i]} - \mathbf{x}_i' \boldsymbol{\beta}_{\lambda}^{[i]} \right)^2$$

where $(\beta_{0(\lambda)}^{[i]}, \boldsymbol{\beta}_{\lambda}^{[i]})$ minimizes $\text{PLS}(\beta_0, \boldsymbol{\beta})$ holding out the i -th data point.

Letting $\hat{\mathbf{y}}_{\lambda} = \mathbf{H}_{\lambda} \mathbf{y} = \{\hat{y}_{i(\lambda)}\}_{n \times 1}$, where $\hat{y}_{i(\lambda)} = \hat{\beta}_{0(\lambda)} + \mathbf{x}_i' \hat{\boldsymbol{\beta}}_{\lambda}$

$$\text{OCV}(\lambda) = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_{i(\lambda)}}{1 - h_{ii(\lambda)}} \right)^2$$

where $h_{ii(\lambda)}$ is the i -th diagonal of $\mathbf{H}_{\lambda} = \frac{1}{n} \mathbf{1}_n \mathbf{1}_n' + \mathbf{X}_c (\mathbf{X}_c' \mathbf{X}_c + \lambda \mathbf{I}_p)^{-1} \mathbf{X}_c'$

Generalized Cross-Validation

Using the OCV to select λ , the influence of each observation depends on the leverage score $h_{ii(\lambda)}$, which could be quite different.

To adjust for this, we could consider a weighted OCV criterion

$$\text{WOCV}(\lambda) = \frac{1}{n} \sum_{i=1}^n w_i \left(\frac{y_i - \hat{y}_{i(\lambda)}}{1 - h_{ii(\lambda)}} \right)^2$$

where $w_i > 0$ is the weight assigned to the i -th observation.

Setting $w_i = \frac{(1-h_{ii(\lambda)})^2}{[n^{-1}\text{tr}(\mathbf{I}-\mathbf{H}_\lambda)]^2}$ replaces each $h_{ii(\lambda)}$ with its average value, resulting in the Generalized Cross-Validation (GCV) (Golub et al., 1979) criterion

$$\text{GCV}(\lambda) = \frac{\frac{1}{n} \|(\mathbf{I} - \mathbf{H}_\lambda)\mathbf{y}\|^2}{[1 - \text{tr}(\mathbf{H}_\lambda)/n]^2} = \frac{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_{i(\lambda)})^2}{(1 - \nu_\lambda/n)^2}$$

where $\nu_\lambda = \text{tr}(\mathbf{H}_\lambda)$ is the effective degrees of freedom.

LASSO (Least Absolute Shrinkage and Selection Operator)

Shrinkage and Selection Simultaneously

Ridge regression shrinks and stabilizes coefficient estimates, but (for $\lambda < \infty$) all regression coefficients remain non-zero.

- Only useful for shrinkage and stabilization
- Not useful for selecting relevant predictors

Ridge regression will not help you determine a parsimonious model.

Lasso (Tibshirani, 1996) does the shrinkage and the selection simultaneously.

Penalized Least Squares Problem

Lasso regression minimizes the penalized least squares (PLS) function

$$\begin{aligned}\text{PLS}_1(\beta_0, \boldsymbol{\beta}) &= \text{OLS}(\beta_0, \boldsymbol{\beta}) + \lambda P_1(\boldsymbol{\beta}) \\ &= \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p |\beta_j|\end{aligned}\tag{3}$$

where $P_1(\boldsymbol{\beta}) = \sum_{j=1}^p |\beta_j|$ and $\lambda > 0$ is a regularization parameter.

Obtaining the lasso estimate $\hat{\boldsymbol{\beta}}_\lambda$ is a convex optimization problem

- least angle regression algorithm (`lars` R package)
- cyclical coordinate descent algorithm (`glmnet` R package)

Comparing Lasso and Ridge

For lasso, we still have:

- As $\lambda \rightarrow 0$, we have $\hat{\beta}_\lambda \rightarrow \hat{\beta}$, where $\hat{\beta}$ is OLS estimate.
- As $\lambda \rightarrow \infty$, we have $\hat{\beta}_\lambda \rightarrow \mathbf{0}$, which is a vector of zeros.

As the ridge regularization parameter increases, all regression coefficients remain non-zero.

As the lasso regularization parameter increases, some regression coefficients remain non-zero and some will go to zero.

K -Fold Cross-Validation

To choose λ , we can use K -fold CV procedure:

- ➊ split sample of data into $K > 1$ folds
 - (a) let $(\mathbf{y}_k, \mathbf{X}_k)$ denote k -th fold's data
 - (b) let $(\mathbf{y}_{[k]}, \mathbf{X}_{[k]})$ denote data holding out the k -th fold
- ➋ For each fold $k = 1, \dots, K$
 - (a) fit model using $(\mathbf{y}_{[k]}, \mathbf{X}_{[k]})$ to estimate $(\hat{\beta}_{0(\lambda)}^{[k]}, \hat{\beta}_{\lambda}^{[k]})$
 - (b) define CV loss for k -th fold as $E_k(\lambda) = \|\mathbf{y}_k - \hat{\beta}_{0(\lambda)}^{[k]} \mathbf{1}_{n_k} - \mathbf{X}_k \hat{\beta}_{\lambda}^{[k]}\|^2$
- ➌ Choose λ that minimizes $CV(\lambda) = \frac{1}{K} \sum_{k=1}^K E_k(\lambda)$

We do this for a sequence of λ values (typically with $K = 5$ or $K = 10$).

Elastic Net

Shrinkage, Selection, and More

The lasso performs well in many situations, but...

- When $p > n$ the lasso can only retain n non-zero coefficients
- When $n > p$ and predictors are highly correlated, ridge is better
- When several predictors are highly correlated, lasso tends to select one predictor and ignore the other (correlated) predictors

Elastic net (Zou & Hastie, 2005) combines the lasso (L_1) and ridge (L_2) penalties to address these issues.

Penalized Least Squares Problem

Elastic net regression considers the same multiple regression model:

$$\mathbf{y} = \mathbf{1}_n\beta_0 + \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon} \quad \text{with} \quad (\mathbf{y}|\mathbf{X}) \sim (\mathbf{1}_n\beta_0 + \mathbf{X}\boldsymbol{\beta}, \sigma^2\mathbf{I}_n)$$

Elastic net minimizes the penalized least squares (PLS) function

$$\text{PLS}_\alpha(\beta_0, \boldsymbol{\beta}) = \frac{1}{2n} \text{OLS}(\beta_0, \boldsymbol{\beta}) + \lambda \text{P}_\alpha(\boldsymbol{\beta}) \quad (4)$$

where $\text{P}_\alpha(\boldsymbol{\beta}) = \sum_{j=1}^p \left\{ \frac{1}{2}(1 - \alpha)\beta_j^2 + \alpha|\beta_j| \right\}$, $\lambda > 0$, and $\alpha \in [0, 1]$ controls the influence of the L_1 and L_2 penalties.

- Set $\alpha = 0$ for ridge and $\alpha = 1$ for lasso
- Set $0 < \alpha < 1$ for combination of ridge and lasso (i.e., elastic net)

Cyclical Coordinate Descent

The Cyclical Coordinate Descent (CCD) algorithm is typically used to estimate the elastic net (and lasso) parameters (Friedman et al., 2010).

Note that we can write

$$\text{PLS}_\alpha = \frac{1}{2n} \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p \left\{ \frac{1}{2} (1 - \alpha) \beta_j^2 + \alpha |\beta_j| \right\}$$

Taking the derivative of PLS_α with respect to β_j gives

$$\frac{\partial \text{PLS}_\alpha}{\partial \beta_j} = -\frac{1}{n} \sum_{i=1}^n x_{ij} \{ y_i - \beta_0 - \sum_{k=1}^p \beta_k x_{ik} \} + \lambda \{ (1 - \alpha) \beta_j + \alpha \text{sign}(\beta_j) \}$$

Cyclical Coordinate Descent (continued)

This implies that the update for β_j is of the form

$$\hat{\beta}_j = \frac{S(\frac{1}{n} \sum_{i=1}^n x_{ij}(y_i - \hat{y}_{i[j]}), \lambda\alpha)}{1 + \lambda(1 - \alpha)}$$

where

- $S(a, b) = \text{sign}(a)(|a| - b)_+$ with $(\cdot)_+ = \max(\cdot, 0)$
 S is the **soft-thresholding operator**
- $\hat{y}_{i[j]} = \hat{\beta}_{0(\lambda)} + \sum_{k \neq j} \hat{\beta}_{k(\lambda)} x_{ik}$
- Note that $n^{-1} \sum_{i=1}^n x_{ij}^2 = 1$ if the data are standardized
 (this is where the **1** comes from)

CCD algorithm repeats the update for $j = 1, \dots, p$ until convergence.

Choosing λ and α via K -Fold Cross-Validation

For the elastic net, there are two parameters to tune via K -fold CV

- Pick a grid of λ and α values
- Perform K -fold CV for each (λ, α) pair
- Choose the (λ, α) pair that minimizes CV loss

Unfortunately, the `cv.glmnet` does not directly tune α for you

- Need to pick a sequence of α and call `cv.glmnet` for each value
- Important to input fold identification (`foldid`) so fold assignments are comparable for different α

Smoothing Spline ANOVA

Parametric versus Nonparametric Regression

The GLM is a form of **parametric regression**, where the relationship between Y and $X_1, \dots, X_p$ has some predetermined form.

- Parameterizes relationship between Y and $X_1, \dots, X_p$
- Then estimates the specified parameters
- Great if you know the form of the relationship (e.g., linear)

In contrast, **nonparametric regression** tries to estimate the form of the relationship between Y and $X_1, \dots, X_p$.

- No predetermined form for relationship between Y and $X_1, \dots, X_p$
- Great for discovering relationships and building prediction models

Smoothing Spline ANOVA

A **smoothing spline analysis of variance (SSANOVA)** model is a statistical framework for estimating functional relationships.

- See Gu (2013), Gu and Wahba (1991), Helwig and Ma (2015), Kim and Gu (2004), Ma et al. (2015), Wahba (1990)

An SSANOVA model has the form

$$y_i = \eta(x_{i1}, \dots, x_{ip}) + \epsilon_i$$

for $i = 1, \dots, n$ where

- y_i is the observed response variable for the i -th subject
- x_{ij} is the j -th observed predictor for the i -th subject
- η is an unknown function relating Y and $X_1, \dots, X_p$
- $\epsilon_i \stackrel{\text{iid}}{\sim} (0, \sigma^2)$ is independent, identically distributed measurement error

Penalized Least Squares Problem

To estimate η , we minimize the **penalized least squares** functional

$$\frac{1}{n} \sum_{i=1}^n (y_i - \eta(\mathbf{x}_i))^2 + \lambda J_{\boldsymbol{\theta}}(\eta) \quad (5)$$

where

- $\mathbf{x}_i = (x_{i1}, \dots, x_{ip})'$ is the predictor vector
- $J_{\boldsymbol{\theta}}(\cdot) = \sum_{k=1}^s \theta_s J_s(\cdot)$ is a non-negative **penalty functional**
- λ and $\boldsymbol{\theta} = (\theta_1, \dots, \theta_s)'$ are non-negative **smoothing parameters**¹

Generalized Cross-Validation criterion (Craven & Wahba, 1979) can be used to estimate the smoothing parameters.

¹Global smoothing parameter λ controls overall influence of smoothness penalty, whereas local smoothing parameters in $\boldsymbol{\theta}$ control influence of marginal penalties.

Penalty Functionals and Spline Types

The form of J is determined by the type of spline being fit.

- Depends on predictors in the model

m -th order **polynomial smoothing spline** with $x \in [0, 1]$

- $J(\eta) = \int_0^1 [\ddot{\eta}^m(x)]^2 dx$ with $\ddot{\eta}^m$ denoting the m -th derivative of η
- Setting $m = 2$ produces cubic spline

Nominal smoothing spline with $x \in \{1, \dots, g\}$

- $J(\eta) = \sum_{x=1}^g [\eta(x) - \bar{\eta}]^2$ with $\bar{\eta} \equiv \sum_{x=1}^g \eta(x)/g$
- Nominal spline is a shrinkage estimate

Ordinal smoothing spline with $x \in \{1 < \dots < g\}$

- $J(\eta) = \sum_{x=2}^g [\eta(x) - \eta(x-1)]^2$
- Ordinal spline penalizes differences between adjacent levels

Multivariate Smoothing via Tensor Product Splines

Tensor product smoothing splines allow for flexible smoothing by introducing additional smoothing parameters to the model.

- Ideal for covariates with **incomparable** units of measurement

Tensor product smoothing spline penalty with $x_1 \in [0, 1]$ and $x_2 \in \{1, \dots, g\}$

$$J(\eta) = \theta_1 J_1(\eta) + \theta_2 J_2(\eta) + \theta_3 x_1 J_2(\eta) + \theta_4 J_1(\eta) J_2(\eta)$$

where

- J_1 and J_2 are the marginal penalties for x_1 and x_2 , respectively
- $\{\theta_k\}_{k=1}^4$ are the additional smoothing parameters

SSANOVA Model Building

With $p > 1$ predictors, we need to specify the form of the function.

For example, with $p = 2$ predictors we have two possible models

additive : $\eta(x_1, x_2) = \eta_0 + \eta_1(x_1) + \eta_2(x_2)$

interactive : $\eta(x_1, x_2) = \eta_0 + \eta_1(x_1) + \eta_2(x_2) + \eta_{12}(x_1, x_2)$

where

- η_0 is an analog of β_0 (unknown regression intercept)
- η_1 is the unknown main effect function for the first predictor
- η_2 is the unknown main effect function for the second predictor
- η_{12} is the unknown interaction effect function

Fitted Values and Smoothing Matrix

Given $\boldsymbol{\lambda} \equiv \{\lambda_k\}_{s \times 1}$ with $\lambda_k \equiv \lambda/\theta_k$, the fitted values are given by

$$\begin{aligned}\hat{\mathbf{y}} &= \mathbf{K}\hat{\mathbf{d}} + \mathbf{J}_{\boldsymbol{\theta}}\hat{\mathbf{c}} \\ &= \mathbf{S}_{\boldsymbol{\lambda}}\mathbf{y}\end{aligned}\tag{6}$$

where

- $\mathbf{K}$ is $n \times m$ null space basis function matrix
- $\mathbf{J}_{\boldsymbol{\theta}}$ is $n \times q$ contrast space basis function matrix
- $\hat{\mathbf{d}}$ and $\hat{\mathbf{c}}$ are estimated basis function coefficients

Note that $\mathbf{S}_{\boldsymbol{\lambda}}$ is the *smoothing matrix*, which

- is an $n \times n$ symmetric matrix
- depends on the chosen smoothing parameters in $\boldsymbol{\lambda}$

Gaussian Process Definition

A **Gaussian process** is a stochastic process $\{\eta(\mathbf{x}) : \mathbf{x} \in \mathcal{X}\}$ such that $\eta(\mathbf{x}) \sim N(\mu_x, \sigma_x^2)$ for all $\mathbf{x} \in \mathcal{X}$ where

- $\mu_x = E(\eta(\mathbf{x}))$ is the mean function
- $\gamma_{x,x'} = Cov(\eta(\mathbf{x}), \eta(\mathbf{x}'))$ is the covariance function
- $\sigma_x^2 = Cov(\eta(\mathbf{x}), \eta(\mathbf{x}))$ is the variance function

Note $\eta(\mathbf{x})$ is a random variable that is normally distributed for all $\mathbf{x} \in \mathcal{X}$

- Use the notation $\eta(\mathbf{x}) \sim N(\mu_x, \sigma_x^2)$ for all $\mathbf{x} \in \mathcal{X}$
- Mean and variance differ for each $\mathbf{x} \in \mathcal{X}$

Bayesian Interpretation of Smoothing Spline

Let $\eta = \eta_n + \eta_c$ denote the null and contrast space functions and assume the following prior distributions:

- η_n has a diffuse (vague) prior with mean zero
- η_c is a zero mean Gaussian process with covariance function proportional to ρ_c

Using the above prior assumptions and assuming $\epsilon_i \stackrel{\text{iid}}{\sim} N(0, \sigma^2)$

- $\hat{\eta}$ can be interpreted as posterior mean of η given data $\mathbf{y}$
- we can derive posterior variance $Var(\eta|\mathbf{y})$

Bayesian Confidence Intervals

Using the Bayesian interpretation, we can form confidence intervals

$$\hat{\eta}(\mathbf{x}) \pm Z_{\alpha/2} \sqrt{\text{Var}(\eta|\mathbf{y})}$$

where $Z_{\alpha/2}$ is critical value from standard normal distribution.

Bayesian CIs have approximate “across-the-function coverage” when the smoothing parameters are selected according to GCV.

- On average contain $100(1 - \alpha)\%$ of true function realizations
- See Wahba (1983), Nychka (1988), and Gu & Wahba (1993)

Student Performance Example

Data Description

Math performance data from Portuguese secondary students.

- Collected by Paulo Cortez (Cortez & Silva, 2008)
- Available from UCI Machine Learning Repository (Lichman, 2013)

Have math exam scores for $n = 395$ students, along with several possible predictor variables (see next slide).

Want to predict score on the first math exam of the year (G1).

Predictor Variables

Table 1: Predictor variables for math performance example.

Variable	Type	Range/Levels
Student's School (school)	binary	1=Gabriel Pereira, 0=Mousinho da Silveira
Student's Sex (sex)	binary	1=male, 0=female
Educational Support (famsup)	binary	1=yes, 0=no
Extra Paid Classes (paid)	binary	1=yes, 0=no
Extra-Curricular Activities (activities)	binary	1=yes, 0=no
Attended Nursery School (nursery)	binary	1=yes, 0=no
Student's Age (age)	integer	15, 16, ..., 22
Number of Failures (failures)	integer	0, 1, 2, 3
Number of Absences (absences)	integer	0, 1, ..., 75
Mother's Education (Medu)	ordinal	none, <4th, 5th-9th, secondary, higher
Travel Time to School (traveltime)	ordinal	<15 min, 15-30 min, 30-60 min, >60 min
Study Time per Week (studytime)	ordinal	<2 hrs, 2-5 hrs, 5-10 hrs, >10 hrs
Goes Out with Friends (goout)	ordinal	1=very low, ..., 5=very high
Weekend Alcohol Consumption (Walc)	ordinal	1=very low, ..., 5=very high
Health Status (health)	ordinal	1=very bad, ..., 5=very good

Data Preprocessing

```
### load data
mypath <- "~/Desktop/"
datapath <- paste0(mypath,"student-mat.csv")
student <- read.table(datapath, sep=";", header=TRUE)

### recode binary predictors
student$school <- ifelse(student$school=="GP",1,0)
student$sex <- ifelse(student$sex=="M",1,0)
student$famsup <- ifelse(student$famsup=="yes",1,0)
student$paid <- ifelse(student$paid=="yes",1,0)
student$activities <- ifelse(student$activities=="yes",1,0)
student$nursery <- ifelse(student$nursery=="yes",1,0)

### declare relevant variables as factors
student$studytime <- factor(student$studytime)
student$Walc <- factor(student$Walc)
student$Medu <- factor(student$Medu)
student$travelttime <- factor(student$travelttime)
student$goout <- factor(student$goout)
student$health <- factor(student$health)
```

Ordinary Least Squares Regression

```
### define formula for linear regression model
form <- as.formula(
  G1 ~ school + sex + famsup + paid + activities + nursery +
    age + failures + absences + Medu + traveltime +
    studytime + goout + Walc + health
)

### fit linear regression model via OLS
linmod = lm(form, data=student)
lincoef = coef(linmod)
```

Ordinary Least Squares Regression (continued)

```
> round(summary(linmod)$coef[1:16,],2)
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	12.57	3.34	3.77	0.00
school	0.10	0.56	0.17	0.87
sex	1.12	0.35	3.17	0.00
famsup	-0.87	0.35	-2.52	0.01
paid	-0.09	0.35	-0.26	0.80
activities	-0.01	0.32	-0.03	0.97
nursery	0.13	0.40	0.34	0.74
age	0.06	0.14	0.42	0.68
failures	-1.31	0.23	-5.73	0.00
absences	0.01	0.02	0.51	0.61
Medu1	-2.40	1.84	-1.30	0.19
Medu2	-2.11	1.83	-1.15	0.25
Medu3	-1.91	1.83	-1.05	0.30
Medu4	-1.06	1.84	-0.57	0.57
traveltime2	-0.32	0.36	-0.89	0.38
traveltime3	0.69	0.70	0.98	0.33

```
> round(summary(linmod)$coef[17:32,],2)
```

	Estimate	Std. Error	t value	Pr(> t)
traveltime4	-1.36	1.20	-1.13	0.26
studytime2	0.32	0.40	0.81	0.42
studytime3	1.72	0.55	3.13	0.00
studytime4	1.41	0.71	2.00	0.05
goout2	0.05	0.72	0.07	0.95
goout3	-0.38	0.71	-0.54	0.59
goout4	-0.63	0.74	-0.85	0.40
goout5	-0.83	0.81	-1.02	0.31
Walc2	0.30	0.43	0.71	0.48
Walc3	0.12	0.45	0.26	0.79
Walc4	-0.96	0.57	-1.69	0.09
Walc5	-0.03	0.76	-0.03	0.97
health2	-0.53	0.65	-0.82	0.41
health3	-1.05	0.57	-1.84	0.07
health4	-0.99	0.60	-1.64	0.10
health5	-0.79	0.53	-1.49	0.14

Ridge Regression

```
### load MASS library  
library(MASS)
```

```
### fit ridge regression model using GCV to select lambda  
ridgemod = lm.ridge(form, data=student, lambda=seq(0,500,length=5000))  
plot(ridgemod$lambda,ridgemod$GCV,xlab=expression(lambda),ylab="GCV")  
gcvmin = which.min(ridgemod$GCV)  
ridgecoef = coef(ridgemod)[gcvmin,]
```

Ridge Regression (continued)

```
> round(cbind(lincoef, ridgecoef) [1:16, ], 2)
```

	lincoef	ridgecoef
(Intercept)	12.57	10.97
school	0.10	0.04
sex	1.12	0.63
famsup	-0.87	-0.62
paid	-0.09	0.00
activities	-0.01	0.07
nursery	0.13	0.15
age	0.06	0.02
failures	-1.31	-0.96
absences	0.01	0.00
Medu1	-2.40	-0.59
Medu2	-2.11	-0.31
Medu3	-1.91	-0.14
Medu4	-1.06	0.60
traveltime2	-0.32	-0.26
traveltime3	0.69	0.32

```
> round(cbind(lincoef, ridgecoef) [17:32, ], 2)
```

	lincoef	ridgecoef
traveltime4	-1.36	-0.98
studytime2	0.32	-0.05
studytime3	1.72	0.91
studytime4	1.41	0.73
goout2	0.05	0.31
goout3	-0.38	0.04
goout4	-0.63	-0.27
goout5	-0.83	-0.51
Walc2	0.30	0.19
Walc3	0.12	0.05
Walc4	-0.96	-0.68
Walc5	-0.03	-0.15
health2	-0.53	0.06
health3	-1.05	-0.39
health4	-0.99	-0.32
health5	-0.79	-0.21

Lasso Regression

```
### load glmnet library
library(glmnet)

### build model matrix (with no intercept column)
y = student$G1
X = model.matrix(form, data=student)[,-1]

### create fold assignments for 10-fold CV
set.seed(1)
foldid = sample(rep(1:10, length.out=length(y)))

### 10-fold CV to estimate lambda
cvlasso = cv.glmnet(X, y, foldid=foldid, alpha=1)
lassomin = coef(cvlasso, s="lambda.min")
lassolse = coef(cvlasso, s="lambda.1se")
```

Lasso Regression (continued)

```
rtab = round(cbind(lincoef,ridgecoef,lassomin,lassolse),2)
colnames(rtab) = c("lincoef","ridgecoef","lassomin","lassolse")
```

```
> rtab[1:16,]
16 x 4 sparse Matrix of class "dgCMatrix"
      lincoef ridgecoef lassomin lassolse
(Intercept) 12.57    10.97    11.13    11.11
school       0.10     0.04     .        .
sex          1.12     0.63     0.58     .
famsup      -0.87    -0.62    -0.55     .
paid        -0.09     0.00     .        .
activities  -0.01     0.07     .        .
nursery      0.13     0.15     .        .
age          0.06     0.02     .        .
failures    -1.31    -0.96    -1.24    -1.01
absences     0.01     0.00     .        .
Medu1       -2.40    -0.59    -0.21     .
Medu2       -2.11    -0.31     .        .
Medu3       -1.91    -0.14     .        .
Medu4       -1.06     0.60     0.74     0.31
traveltime2 -0.32    -0.26    -0.07     .
traveltime3  0.69     0.32     .        .
```

```
> rtab[17:32,]
16 x 4 sparse Matrix of class "dgCMatrix"
      lincoef ridgecoef lassomin lassolse
traveltime4 -1.36    -0.98    -0.57     .
studytime2   0.32    -0.05     .        .
studytime3   1.72     0.91     0.92    0.18
studytime4   1.41     0.73     0.41     .
goout2        0.05     0.31     0.17     .
goout3       -0.38     0.04     .        .
goout4       -0.63    -0.27    -0.09     .
goout5       -0.83    -0.51    -0.30     .
Walc2         0.30     0.19     .        .
Walc3         0.12     0.05     .        .
Walc4        -0.96    -0.68    -0.62     .
Walc5        -0.03    -0.15     .        .
health2      -0.53     0.06     .        .
health3      -1.05    -0.39    -0.04     .
health4      -0.99    -0.32     .        .
health5      -0.79    -0.21     .        .
```

Elastic Net Regression

```
### load glmnet library
library(glmnet)

### build model matrix (with no intercept column)
y = student$G1
X = model.matrix(form, data=student)[,-1]

### create fold assignments for 10-fold CV
set.seed(1)
foldid = sample(rep(1:10, length.out=length(y)))

### 10-fold CV to estimate lambda and alpha
alphaseq = seq(0,1,length=11)
cvlist = vector("list",length(alphaseq))
for(k in 1:length(alphaseq)){
  cvlist[[k]] = cv.glmnet(X, y, foldid=foldid, alpha=alphaseq[k])
}
mincv = sapply(cvlist, function(x) min(x$cvm))
minid = which.min(mincv)
```

Elastic Net Regression (continued)

```
### which alpha is best?
```

```
> minid
```

```
[1] 11
```

```
> alphaseq[minid]
```

```
[1] 1
```

```
### compare to lasso coefficients (alpha=1 is lasso)
```

```
> sum( ( lassomin - coef(cvlist[[minid]],s="lambda.min") )^2 )
```

```
[1] 0
```

```
> sum( ( lassolse - coef(cvlist[[minid]],s="lambda.1se") )^2 )
```

```
[1] 0
```

Smoothing Spline ANOVA

```
### load bigsplines library
library(bigsplines)

### define variable types for semiparametric regression model
type <- list(school="prm", sex="prm", famsup="prm", paid="prm",
             activities="prm", nursery="prm", age="cub", failures="cub",
             absences="cub", Medu="ord", traveltime="ord",
             studytime="ord", goout="ord", Walc="ord", health="ord")

### fit semiparametric regression model (with all data points as knots)
mymod <- bigssp(form, data=student, type=type, nknots=395, knotcheck=F)
```

Smoothing Spline ANOVA (continued)

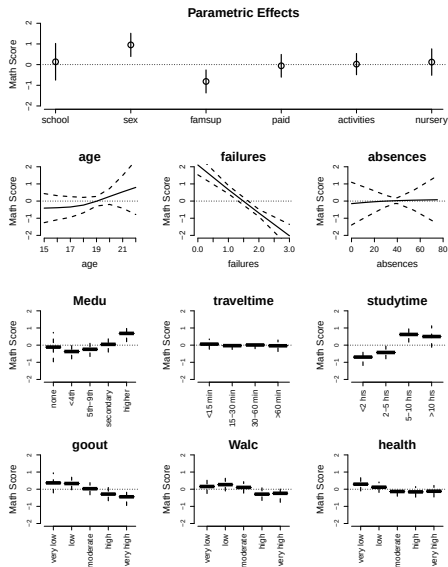


Figure 1: Estimated effects of the different predictor variables on expected math scores with 90% Bayesian confidence intervals (dashed lines) around the predictions.

Summary of Results

Comparing OLS and ridge regression...

- OLS declares a few variables as significant
- OLS tends to overfit (larger coefficients than ridge)

Compared to OLS and ridge, lasso and elastic net...

- Select a sparser model (but which do we choose)?
- Do not provide any inference for free (need bootstrap)

Compared to others, the SSANOVA model...

- Does not require assumption of linear relationships
- Provides Bayesian CIs, which can be used for inference
- Offers a coherent framework for ordinal predictors

References

- Cortez, P. and A. Silva (2008). Using data mining to predict secondary school student performance. In A. Brito and J. Teixeira (Eds.), *Proceedings of 5th Future Business Technology Conference (FUBUTEC 2008)*, pp. 5–12.
- Craven, P. and G. Wahba (1979). Smoothing noisy data with spline functions: Estimating the correct degree of smoothing by the method of generalized cross-validation. *Numerische Mathematik* 31, 377–403.
- Friedman, J., T. Hastie, and R. Tibshirani (2010). Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software* 33, 1–22.
- Golub, G. H., M. Heath, and G. Wahba (1979). Generalized cross-validation as a method for choosing a good ridge parameter. *Technometrics* 21, 215–223.
- Gu, C. (2013). *Smoothing Spline ANOVA Models* (Second ed.). New York: Springer-Verlag.
- Gu, C. and G. Wahba (1991). Minimizing GCV/GML scores with multiple smoothing parameters via the newton method. *SIAM Journal on Scientific and Statistical Computing* 12, 383–398.
- Gu, C. and G. Wahba (1993). Smoothing spline ANOVA with component-wise bayesian “confidence intervals”. *Journal of Computational and Graphical Statistics* 2, 97–117.
- Helwig, N. E. and P. Ma (2015). Fast and stable multiple smoothing parameter selection in smoothing spline analysis of variance models with large samples. *Journal of Computational and Graphical Statistics* 24, 715–732.
- Hoerl, A. and R. Kennard (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics* 12, 55–67.
- Kim, Y.-J. and C. Gu (2004). Smoothing spline gaussian regression: More scalable computation via efficient approximation. *Journal of the Royal Statistical Society, Series B* 66, 337–356.
- Lichman, M. (2013). UCI machine learning repository.
- Ma, P., J. Huang, and N. Zhang (2015). Efficient computation of smoothing splines via adaptive basis sampling. *Biometrika* 102, 631–645.
- Nychka, D. (1988). Bayesian confidence intervals for smoothing splines. *Journal of the American Statistical Association* 83, 1134–1143.
- Tibshirani, R. (1996). Regression and shrinkage via the lasso. *Journal of the Royal Statistical Society, Series B* 58, 267–288.
- Wahba, G. (1983). Bayesian “confidence intervals” for the cross-validated smoothing spline. *Journal of the Royal Statistical Society, Series B* 45, 133–150.
- Wahba, G. (1990). *Spline models for observational data*. Philadelphia: Society for Industrial and Applied Mathematics.
- Zou, H. and T. Hastie (2005). Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society, Series B* 67, 301–320.